

Министерство науки и высшего образования
Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
ИРИТ-РТФ
Школа магистратуры

Лабораторная работа №6

«Введение в многопоточность»

Преподаватель

Архипов Н.А.

Студент

Жунёв А.А.

Группа РИМ-150950

Цель работы

Получить представление о механизме многопоточности в языке программирования Java.

Ссылка на репозиторий: https://github.com/2made1ra/JAVA_SBER_2026

Код лабораторной работы расположен в пакете: java-core-2026/src/lab6

1. Описание задания

В рамках лабораторной работы необходимо выполнить задания для самостоятельной работы из раздела 2:

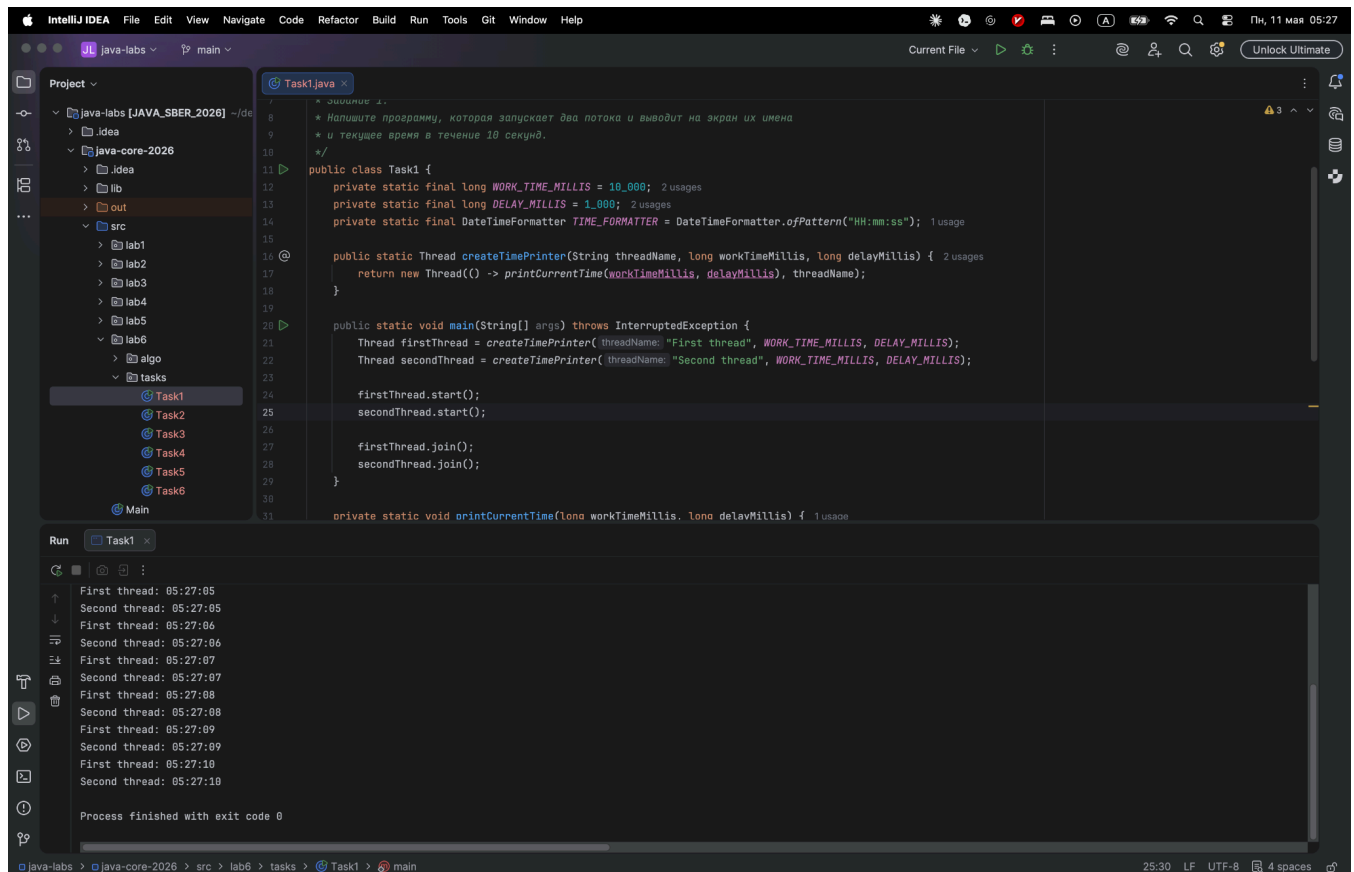
1. Написать программу, которая запускает два потока и выводит на экран их имена и текущее время в течение 10 секунд.
2. Написать программу, которая запускает поток и выводит на экран числа от 1 до 10 с задержкой в 1 секунду между выводами.
3. Написать программу, которая создает два потока, которые выводят на экран числа от 1 до 10. Первый поток должен выводить только четные числа, а второй поток должен выводить только нечетные числа.
4. Написать программу, которая создает 10 потоков и каждый поток выводит на экран свой номер.
5. Написать функцию, которая находит максимальный элемент в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.
6. Написать функцию, которая суммирует элементы в массиве целых чисел при помощи многопоточности. Количество потоков должно быть равно количеству ядер процессора.
7. Решить две алгоритмические задачи. Для выполнения этого пункта были выбраны задачи LeetCode:
 - Valid Parentheses: <https://leetcode.com/problems/valid-parentheses/description/>
 - Merge Two Sorted Lists:
<https://leetcode.com/problems/merge-two-sorted-lists/description/>

2. Задания для самостоятельной работы

Задание 1. Вывод имени потока и текущего времени

Файл с решением: `java-core-2026/src/lab6/tasks/Task1.java`

В классе `Task1` создаются два потока. Каждый поток в течение 10 секунд выводит свое имя и текущее время. Для паузы между выводами используется `Thread.sleep`, а основной поток ожидает завершения рабочих потоков через `join`.



The screenshot displays the IntelliJ IDEA IDE with the file `Task1.java` open. The code defines a class `Task1` with two static threads, `firstThread` and `secondThread`, each printing their names and the current time every 1000 milliseconds for 10 seconds. The `main` method starts both threads and waits for them to finish using `join()`. The Run window at the bottom shows the output of the program, displaying the names of the threads and the current time in HH:mm:ss format, alternating between the two threads.

```
1 // Задание 1.
2 * Напишите программу, которая запускает два потока и выводит на экран их имена
3 * и текущее время в течение 10 секунд.
4 */
5
6 public class Task1 {
7
8     private static final long WORK_TIME_MILLIS = 10_000; // 2 usages
9     private static final long DELAY_MILLIS = 1_000; // 2 usages
10    private static final DateTimeFormatter TIME_FORMATTER = DateTimeFormatter.ofPattern("HH:mm:ss"); // 1 usage
11
12    public static Thread createTimePrinter(String threadName, long workTimeMillis, long delayMillis) { // 2 usages
13        return new Thread(() -> printCurrentTime(workTimeMillis, delayMillis), threadName);
14    }
15
16    public static void main(String[] args) throws InterruptedException {
17        Thread firstThread = createTimePrinter("First thread", WORK_TIME_MILLIS, DELAY_MILLIS);
18        Thread secondThread = createTimePrinter("Second thread", WORK_TIME_MILLIS, DELAY_MILLIS);
19
20        firstThread.start();
21        secondThread.start();
22
23        firstThread.join();
24        secondThread.join();
25    }
26
27    private static void printCurrentTime(long workTimeMillis, long delayMillis) { // 1 usage
28        // ...
29    }
30
31 }
```

Run Task1 x

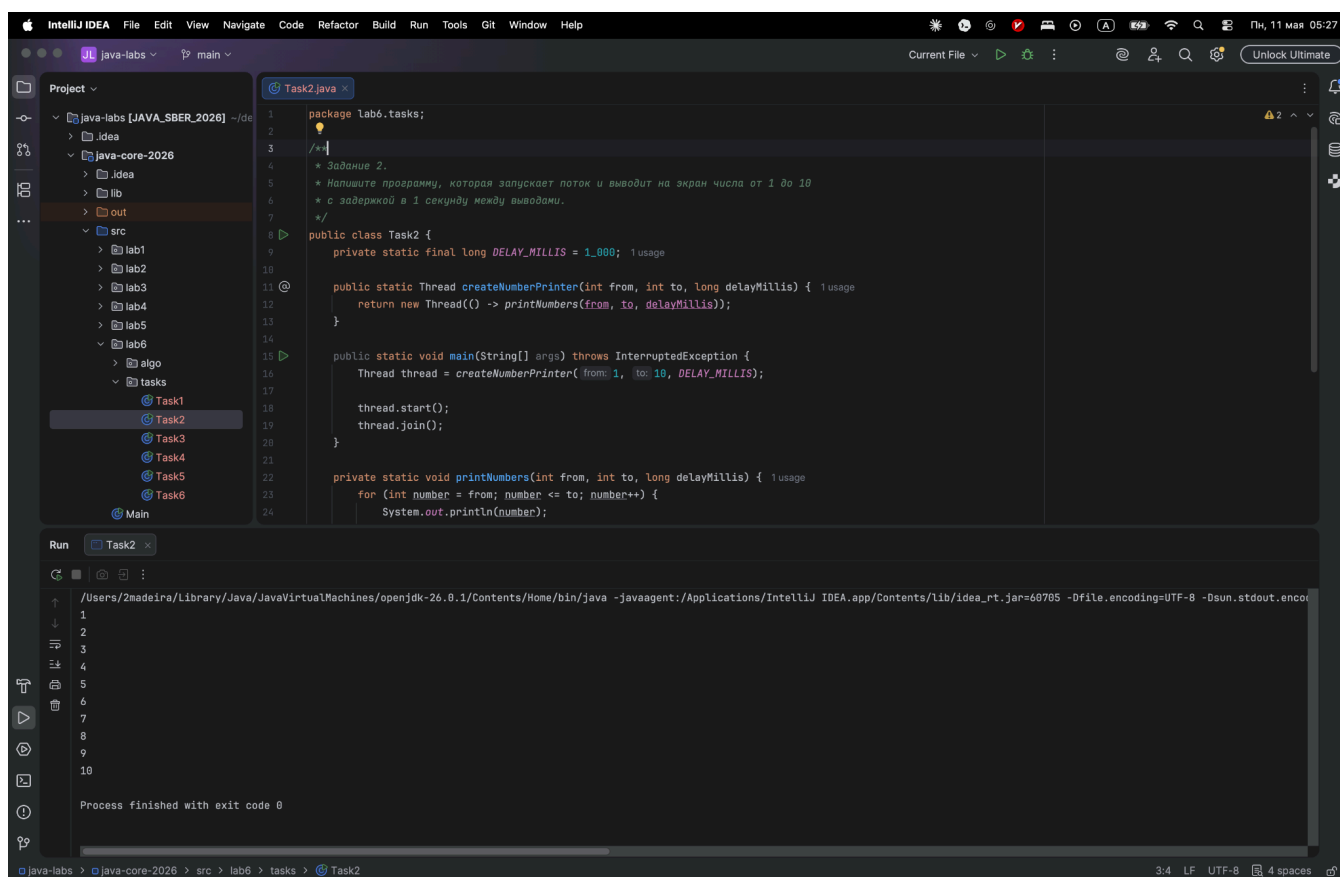
```
First thread: 05:27:05
Second thread: 05:27:05
First thread: 05:27:06
Second thread: 05:27:06
First thread: 05:27:07
Second thread: 05:27:07
First thread: 05:27:08
Second thread: 05:27:08
First thread: 05:27:09
Second thread: 05:27:09
First thread: 05:27:10
Second thread: 05:27:10

Process finished with exit code 0
```

Задание 2. Вывод чисел от 1 до 10 в отдельном потоке

Файл с решением: `java-core-2026/src/lab6/tasks/Task2.java`

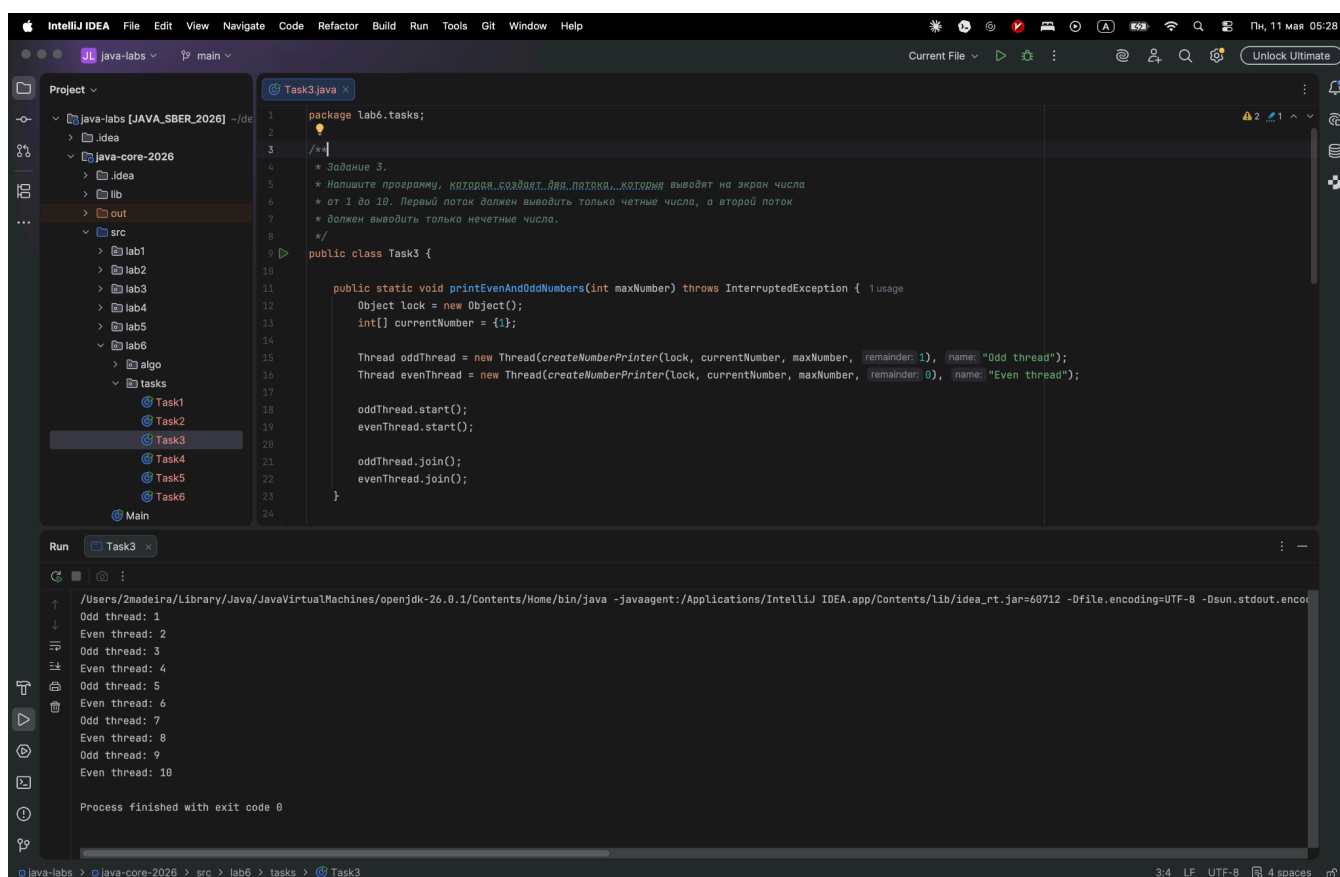
В классе `Task2` создается один поток, который выводит числа от 1 до 10. Между выводами чисел используется задержка в 1 секунду.



Задание 3. Вывод четных и нечетных чисел двумя потоками

Файл с решением: `java-core-2026/src/lab6/tasks/Task3.java`

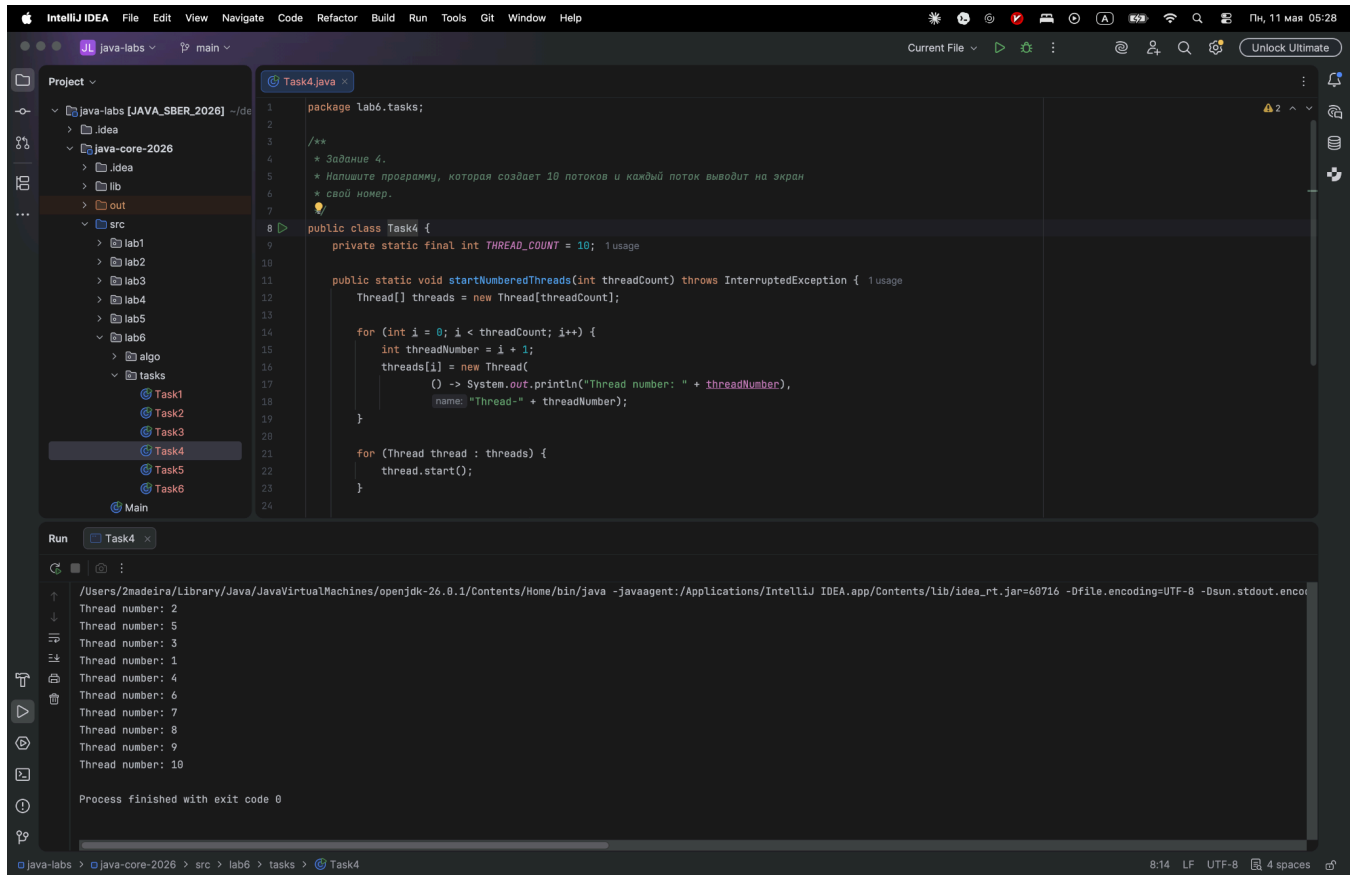
В классе `Task3` реализованы два потока: один выводит нечетные числа, второй выводит четные числа. Для согласованного доступа к текущему числу используется общий объект-блокировка, `wait` и `notifyAll`.



Задание 4. Запуск 10 потоков

Файл с решением: `java-core-2026/src/lab6/tasks/Task4.java`

В классе `Task4` создается массив из 10 потоков. Каждый поток выводит свой порядковый номер, после чего основной поток ожидает завершения всех созданных потоков.



The screenshot displays the IntelliJ IDEA IDE with the file `Task4.java` open. The code defines a class `Task4` with a static final integer `THREAD\_COUNT` set to 10. A static method `startNumberedThreads` is implemented, which creates an array of 10 threads. Each thread, when started, prints its assigned number (from 1 to 10) to the standard output. The main method of the class calls `startNumberedThreads` with the thread count. Below the code editor, the 'Run' window shows the output of the program, displaying the thread numbers in a non-sequential order: 2, 5, 3, 1, 4, 6, 7, 8, 9, and 10. The process concludes with the message 'Process finished with exit code 0'.

```
1 package lab6.tasks;
2
3 /**
4  * Задание 4.
5  * Напишите программу, которая создает 10 потоков и каждый поток выводит на экран
6  * свой номер.
7  */
8
9 public class Task4 {
10     private static final int THREAD_COUNT = 10;
11
12     public static void startNumberedThreads(int threadCount) throws InterruptedException {
13         Thread[] threads = new Thread[threadCount];
14
15         for (int i = 0; i < threadCount; i++) {
16             int threadNumber = i + 1;
17             threads[i] = new Thread(
18                 () -> System.out.println("Thread number: " + threadNumber),
19                 "Thread-" + threadNumber);
20         }
21
22         for (Thread thread : threads) {
23             thread.start();
24         }
25     }
26 }
```

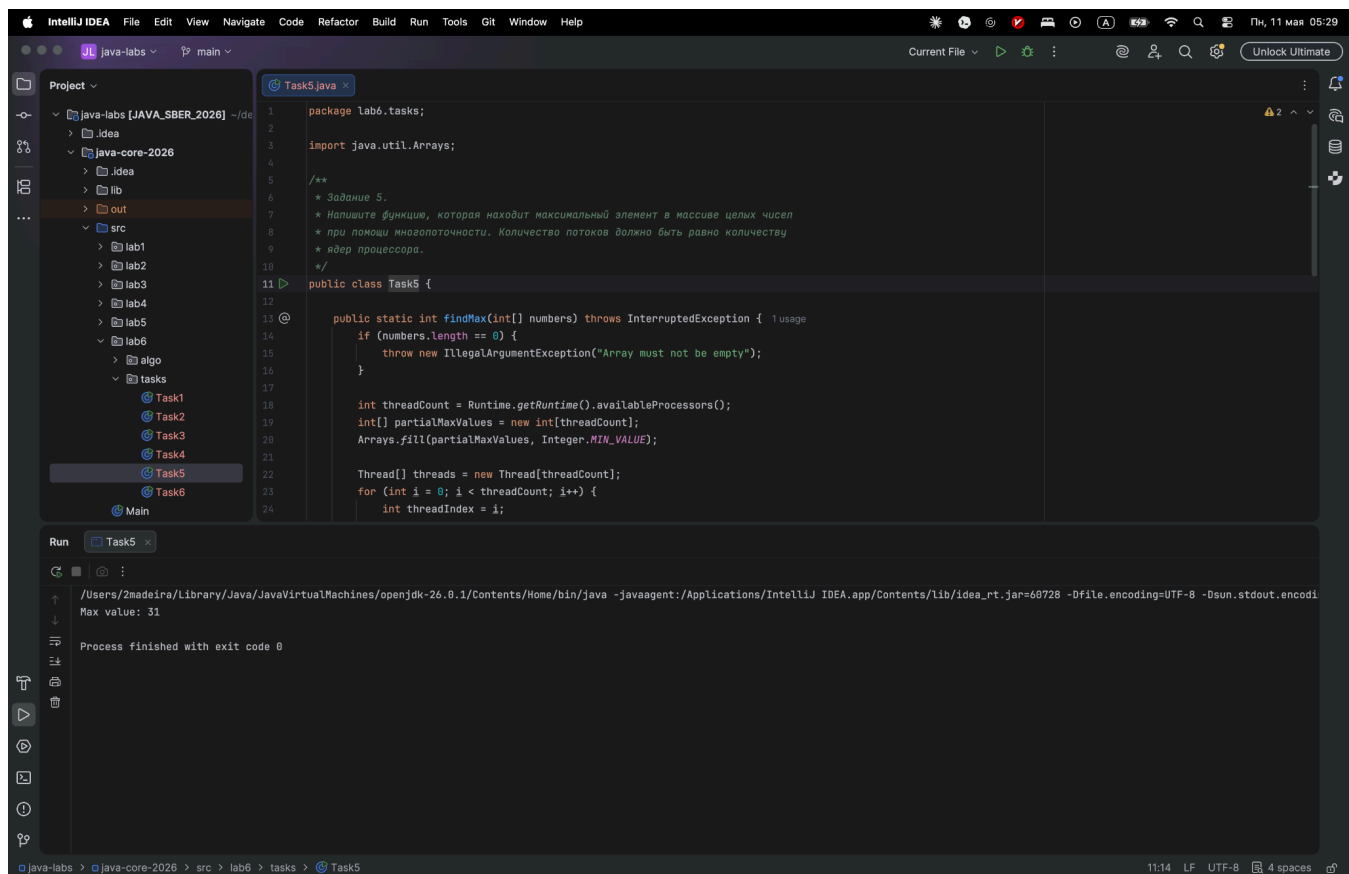
Run Task4

```
/Users/zmadeira/Library/Java/JavaVirtualMachines/openjdk-26.0.1/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/lib/idea_rt.jar=60716 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
Thread number: 2
Thread number: 5
Thread number: 3
Thread number: 1
Thread number: 4
Thread number: 6
Thread number: 7
Thread number: 8
Thread number: 9
Thread number: 10
Process finished with exit code 0
```

Задание 5. Поиск максимального элемента массива с использованием многопоточности

Файл с решением: `java-core-2026/src/lab6/tasks/Task5.java`

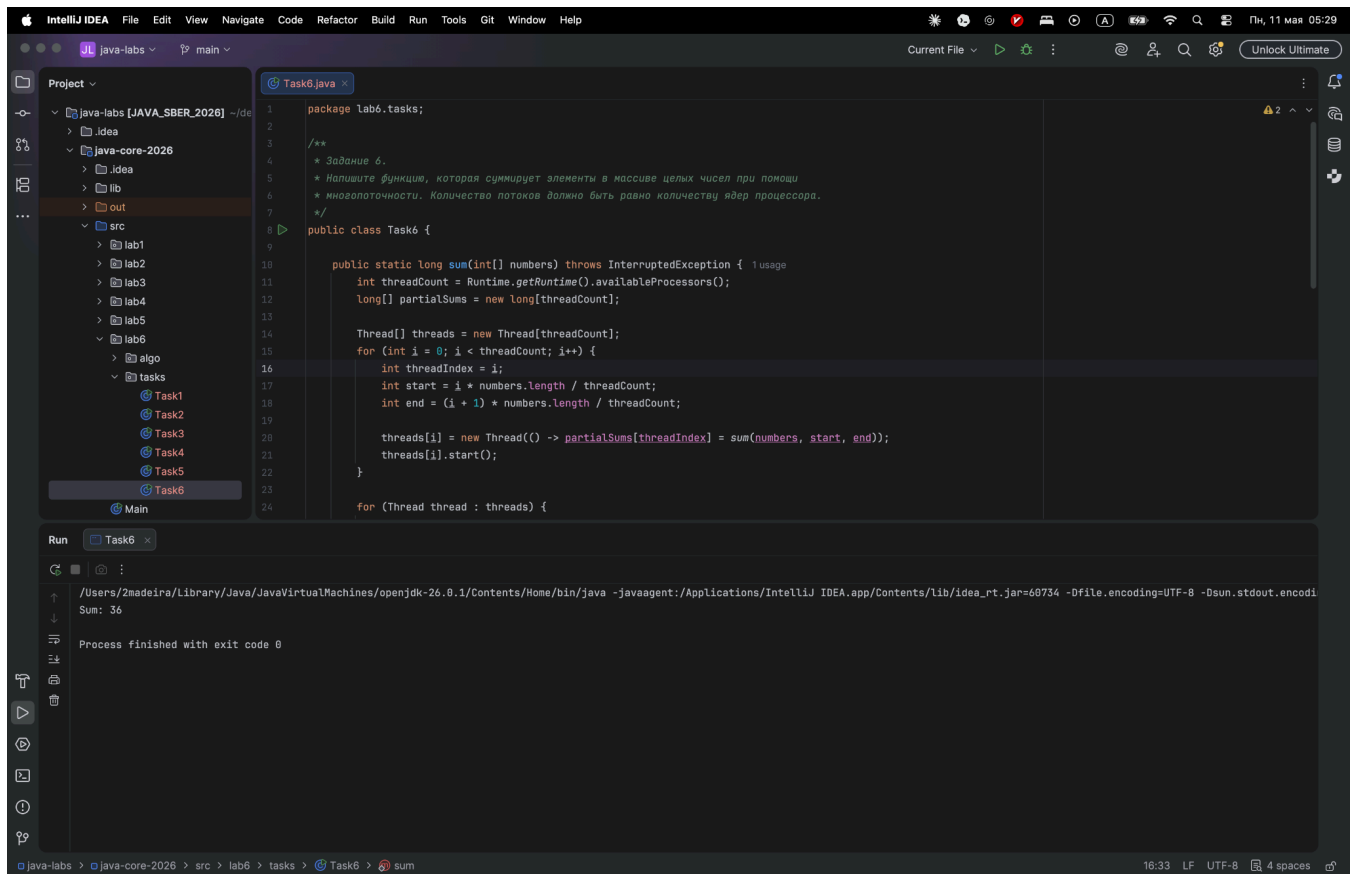
В классе `Task5` реализован метод `findMax`, который делит массив на участки по количеству доступных ядер процессора. Каждый поток ищет максимум на своем участке массива, после чего итоговый максимум находится среди частичных результатов.



Задание 6. Суммирование элементов массива с использованием многопоточности

Файл с решением: `java-core-2026/src/lab6/tasks/Task6.java`

В классе `Task6` реализован метод `sum`, который делит массив на участки по количеству доступных ядер процессора. Каждый поток считает сумму своего участка, после чего частичные суммы объединяются в общий результат.



3. Алгоритмические задания

Задача 7.1. Valid Parentheses

Ссылка на задачу: <https://leetcode.com/problems/valid-parentheses/description/>

Файл с решением: `java-core-2026/src/lab6/algo/task1/Solution.java`

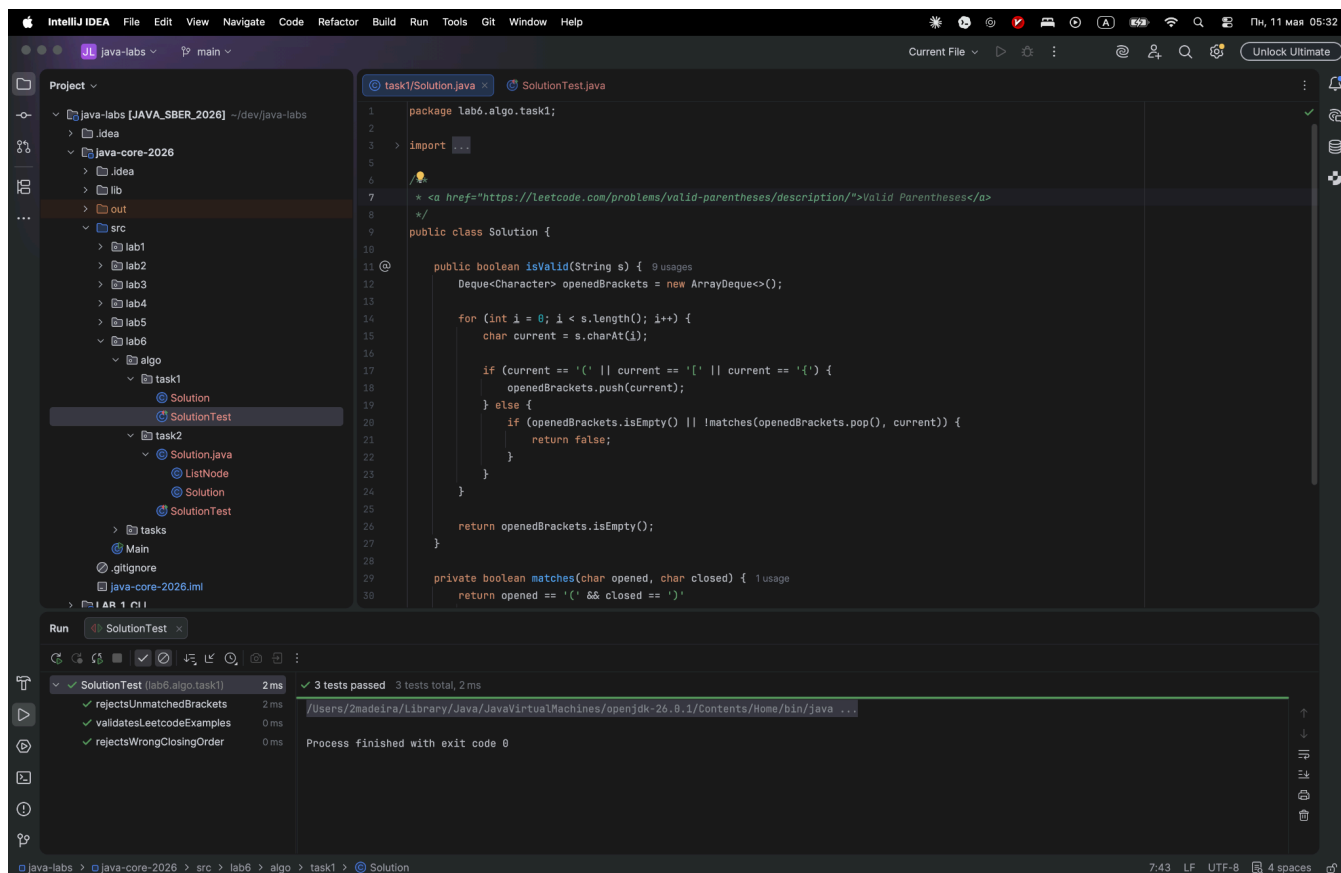
Файл с тестами: `java-core-2026/src/lab6/algo/task1/SolutionTest.java`

В решении используется стек `ArrayDeque`. Открывающие скобки помещаются в стек, а при встрече закрывающей скобки проверяется соответствие последней открытой скобке.

The screenshot displays the LeetCode submission page for the 'Valid Parentheses' problem. The submission is marked as 'Accepted' with 102/102 test cases passed. The runtime is 3ms, which beats 87.77% of other submissions, and the memory usage is 43.30 MB, beating 57.00%. The submitted Java code is as follows:

```
1 public class Solution {
2
3     public boolean isValid(String s) {
4         Deque<Character> openedBrackets = new ArrayDeque<>();
5
6         for (int i = 0; i < s.length(); i++) {
7             char current = s.charAt(i);
8
9             if (current == '(' || current == '[' || current == '{') {
10                 openedBrackets.push(current);
11             } else {
12                 if (openedBrackets.isEmpty() || !matches(openedBrackets.pop(), current)) {
13                     return false;
14                 }
15             }
16         }
17     }
18 }
```

The test results section shows 'Accepted' status with a runtime of 0ms. Five test cases are listed, all of which are passed (Case 1, Case 2, Case 3, Case 4, Case 5). The input for Case 1 is `s = "()"`, the output is `true`, and the expected result is also `true`.



Задача 7.2. Merge Two Sorted Lists

Ссылка на задачу: <https://leetcode.com/problems/merge-two-sorted-lists/description/>

Файл с решением: `java-core-2026/src/lab6/algo/task2/Solution.java`

Файл с тестами: `java-core-2026/src/lab6/algo/task2/SolutionTest.java`

В решении используется вспомогательный фиктивный узел `dummy`, через который последовательно собирается итоговый отсортированный связный список.

Вывод

В ходе лабораторной работы были изучены базовые механизмы многопоточности в Java: создание и запуск потоков, ожидание завершения через `join`, временная остановка выполнения через `sleep`, а также синхронизация потоков с помощью `wait` и `notifyAll`.

Также были реализованы многопоточные функции поиска максимального элемента и суммирования элементов массива, где количество потоков соответствует количеству доступных ядер процессора. В рамках алгоритмической части были решены две задачи LeetCode и подготовлены JUnit-тесты для проверки корректности решений.